Colecțiile din .NET și LINQ — Exerciții

Ștefan · Partea 2 · Exercițiul 0 + 15 exerciții, în consolă · întâi cu bucle, apoi cu LINQ

Unde scrii: în același proiect `Colections`, nu adaugi alt proiect. În rădăcina proiectului, lângă `Program.cs`, creezi fișierul `Etapa2.cs`, cu `static class Exercitii`. Fiecare exercițiu = o metodă `public static void ExN()` cu codul din „În `ExN`" (copiat ca atare), iar metodele din enunț stau lângă ea, în aceeași clasă. În `Main` chemi `Exercitii.ExN();` pentru exercițiul la care lucrezi. Ieșirea trebuie să fie **identică**.
`Etapa2.cs`, sus: `#pragma warning disable CS8600, CS8601, CS8602, CS8603, CS8604, CS8618, CS8625, CS8765` (proiectul tău are `Nullable` activat, iar în C# 7.3 nu există `string?`), apoi `using System;`, `using System.Collections.Generic;`, `using System.Linq;`. Clasele `Elev`, `Client`, `Punct` stau în `Etapa2.cs`, în afara lui `Exercitii`, fără namespace.
Reguli: C# 7.3 și metode existente în 4.x (TEORIE §0) · exercițiile 0–9 **fără LINQ** · 10–14 **doar LINQ**, fără `for` / `foreach` în metode · un exercițiu per commit · pe prima linie din `Main`: `CultureInfo.CurrentCulture = CultureInfo.InvariantCulture;` (cu `using System.Globalization;`), ca zecimalele să iasă cu punct.
Exemplul: `etapa-2-colectii-csharp/ExempluAeroport.cs`, cu 12 secțiuni; vezi TEORIE, primul chenar.

Clasele de care ai nevoie

`Elev` și `Client` stau în `Etapa2.cs`, în afara clasei `Exercitii`; `Catalog()` stă în `Exercitii`.

```
class Elev
{
    public string Nume { get; set; }
    public string Clasa { get; set; }
    public double Medie { get; set; }

    public Elev(string nume, string clasa, double medie)
    {
        Nume = nume;
        Clasa = clasa;
        Medie = medie;
    }
}

class Client
{
    public string Nume { get; set; }
    public int Portii { get; set; }

    public Client(string nume, int portii)
    {
        Nume = nume;
        Portii = portii;
    }
}

static List<Elev> Catalog()
{
    return new List<Elev>
    {
        new Elev("Ana", "12A", 9.50), new Elev("Ion", "11B", 8.20),
```

```
        new Elev("Maria", "12A", 9.80), new Elev("Dan", "11B", 9.10),
        new Elev("Ioana", "10C", 7.40), new Elev("Vlad", "10C", 9.50),
        new Elev("Elena", "12A", 4.90)
    };
}
```

Partea 0 — Puntea: aceiași pași, cu colecțiile din .NET

Citește întâi TEORIE §1. `Ex0()` cheamă pe rând `Ex0a()`, `Ex0b()`, `Ex0c()`, cu `Console.WriteLine("---");` între ele.

0a. Pas 1, pe `List<string>`

Copiezi `DemoLista` din `Program.cs` în `Exercitii` și îi schimbi doar parametrul: `List<string> lista`. `Ex0a` face `DemoLista(new List<string>());`. Rulezi și notezi ce afișează prima linie.

Apoi adaugi metoda de mai jos și înlocuiești în `DemoLista` fiecare `lista + ...` cu `Text(lista) + ...`.

```
static string Text<T>(IEnumerable<T> elemente)
{
    return "[" + string.Join(", ", elemente) + "]";
}
```

Ieșire așteptată:

```
[] Count=0
[a, b, c, d, e] Count=5
True False
[b, D, f] Count=3
2 -1
True False
b;D;f;
b|D|f
[z] z Count=1
```

Gata când: ieșirea e identică, rând cu rând, cu cea de la Pas 1.

Teorie: §1

0b. Pas 5a, pe `Dictionary<string, int>`

Copiezi `Pas5a` în `Ex0b` și `Sortat` în `Exercitii`. Schimbări:

- `DictionarHash<string, int>` → `Dictionary<string, int>`, `TryToTake` → `TryGetValue`;
- pe fiecare rând din ciclu afișezi doar `Count` (nu mai ai câte găleți);
- după blocul `if` cu `"Maria"` adaugi `Console.WriteLine(varste.ContainsKey("Maria"));`;
- `Sortat` devine:

```
static string Sortat<T>(IEnumerable<T> elemente)
{
    List<T> lista = new List<T>(elemente);
    lista.Sort();
    return string.Join(", ", lista);
}
```

Ieșire așteptată:

```
Ana: Count=1
Ion: Count=2
Maria: Count=3
Dan: Count=4
Ioana: Count=5
Vlad: Count=6
Elena: Count=7
30
True False
True False
Maria 17
True
False 0
Ana, Elena, Ioana, Ion, Maria, Vlad
15, 17, 19, 20, 21, 30
```

Teorie: §1, §5

0c. Pas 6, pe colecțiile din .NET

Copiezi `Incearca` și `Pas6` (în `Ex0c`). Schimbări:

- `ListaTablou<int>` → `List<int>`; `ListaInlantuita<int>` → `LinkedList<int>` (umplută cu `AddLast`); `StivaTablou` → `Stack`; `CoadaCirculara` → `Queue`; `DictionarHash` → `Dictionary`;
- un `LinkedList` nu are `RemoveAt`: linia cu `inlantuita.StergeLa(-1)` devine `lista.RemoveAt(-1)`, cu eticheta `"lista.RemoveAt(-1)"`;
- celelalte etichete iau numele metodelor din .NET: `"lista.Insert(5, 6)"`, `"stiva.Pop()"`, `"coada.Peek()"`, `"d.Add(\"Ana\", 18)"`;
- `Console.WriteLine(lista)` și `Console.WriteLine(inlantuita)` devin `Console.WriteLine(Text(lista))`, respectiv `Text(inlantuita)`.

Ieșire așteptată:

```
lista[5] -> ArgumentOutOfRangeException
lista.Insert(5, 6) -> ok
lista.RemoveAt(-1) -> ArgumentOutOfRangeException
stiva.Pop() -> InvalidOperationException
coada.Peek() -> InvalidOperationException
d["Ion"] -> KeyNotFoundException
d.Add("Ana", 18) -> ArgumentException
Collection was modified; enumeration operation may not execute.
[1, 3, 4, 5, 6]
Collection was modified after the enumerator was instantiated.
[1, 2, 3, 4]
[4, 6]
```

Gata când: ieșirile sunt cele de mai sus. În `RASPUNSURI.txt`: pui ieșirea de la Ex0b lângă cea de la Pas 5a și ieșirea de la Ex0c lângă cea de la Pas 6. Care rânduri diferă (și față de ce afișează colecțiile tale acum)? Pentru fiecare, ce face .NET altfel și de ce?

Teorie: §1, §4, §7

Partea 1 — List

1. Dublează numerele pare

Metoda: `static void DubleazaPare(List<int> lista)`: după fiecare număr par inserează încă o copie a lui, **în aceeași listă**.

În `Ex1`:

```
List<int> numere = new List<int> { 1, 2, 3, 4, 4, 7 };
DubleazaPare(numere);
Console.WriteLine(string.Join(" ", numere));
List<int> faraPare = new List<int> { 1, 3, 5 };
DubleazaPare(faraPare);
Console.WriteLine(string.Join(" ", faraPare));
```

Ieșire așteptată:

```
1 2 2 3 4 4 4 4 7
1 3 5
```

Teorie: §3, §4

Indiciu: dacă programul nu se mai termină, citește al treilea punct din §4.

2. Fără dubluri, în ordinea primei apariții

`static List<string> FaraDubluri(List<string> nume)` : o listă nouă, fiecare nume o singură dată, în ordinea în care a apărut prima oară. Literele mari contează: „Ana” ≠ „ana”.

În `Ex2` :

```
List<string> nume = new List<string>
    { "Ana", "Ion", "ana", "Ion", "Maria", "Ana" };
Console.WriteLine(string.Join(", ", FaraDubluri(nume)));
```

Ieșire așteptată:

```
Ana, Ion, ana, Maria
```

Teorie: §6 · folosești un `HashSet` pentru „am văzut deja”, nu `List.Contains`

Partea 2 — Dictionary

3. De câte ori apare fiecare literă

`static Dictionary<char, int> Frecvente(string text)` : doar litere, fără să conteze dacă sunt mari sau mici.

În `Ex3` :

```
Dictionary<char, int> f = Frecvente("Ana are mere!");
List<char> litere = new List<char>(f.Keys);
litere.Sort();
foreach (char c in litere)
{
    Console.WriteLine(c + ": " + f[c]);
}
```

Ieșire așteptată:

```
a: 3
e: 3
m: 1
n: 1
r: 2
```

Teorie: §5, tiparul numărătorului

4. Elevii, pe clase

`static Dictionary<string, List<string>> GrupeazaDupaClasa(List<Elev> elevi)` : clasă → numele elevilor din ea, în ordinea din catalog.

În `Ex4`:

```
Dictionary<string, List<string>> grupe = GrupeazaDupaClasa(Catalog());
List<string> clase = new List<string>(grupe.Keys);
clase.Sort();
foreach (string clasa in clase)
{
    Console.WriteLine(clasa + ": " + string.Join(", ", grupe[clasa]));
}
```

Ieșire așteptată:

```
10C: Ioana, Vlad
11B: Ion, Dan
12A: Ana, Maria, Elena
```

Teorie: §5

Indiciu: la prima întâlnire a unei clase, lista ei încă nu există. Cu `TryGetValue` afli asta și o creezi tu.

5. Agenda, fără excepții

`static bool Adauga(Dictionary<string, string> agenda, string nume, string telefon)`: întoarce `false` și nu schimbă nimic dacă numele există deja.
`static string Cauta(Dictionary<string, string> agenda, string nume)`: întoarce `"necunoscut"` pentru un nume care nu e în agendă. Niciuna dintre ele nu are voie să arunce.

În `Ex5`:

```
Dictionary<string, string> agenda = new Dictionary<string, string>();
Console.WriteLine(Adauga(agenda, "Ana", "0722111222"));
Console.WriteLine(Adauga(agenda, "Ion", "0733444555"));
Console.WriteLine(Adauga(agenda, "Ana", "0700000000"));
Console.WriteLine(Cauta(agenda, "Ana"));
Console.WriteLine(Cauta(agenda, "Maria"));
```

Ieșire așteptată:

```
True
True
False
0722111222
necunoscut
```

Teorie: §5, tabelul cu ce aruncă și ce nu

Partea 3 — HashSet, Queue, Stack

6. Cercurile de la școală

`static HashSet<string> InAmbele(List<string> a, List<string> b)` și `static HashSet<string> DoarLaPrimul(List<string> a, List<string> b)`, cu operațiile de mulțime din §6. Listele primite nu se modifică.

În `Ex6`:

```
List<string> robotica = new List<string> { "Ana", "Ion", "Maria", "Dan" };
List<string> sah = new List<string> { "Ion", "Ioana", "Dan", "Vlad" };
List<string> ambele = new List<string>(InAmbele(robotica, sah));
ambele.Sort();
Console.WriteLine(string.Join(", ", ambele));
List<string> doarRobotica = new List<string>(DoarLaPrimul(robotica, sah));
doarRobotica.Sort();
Console.WriteLine(string.Join(", ", doarRobotica));
```

Ieșire așteptată:

```
Dan, Ion
Ana, Maria
```

Teorie: §6

7. Coada la cantină

`static List<string> OrdineTerminare(Queue<Client> coada)`: fiecare client din față primește **o** porție. Dacă mai are nevoie de porții, se așază din nou la coadă, la final. Întoarce numele, în ordinea în care au terminat.

În `Ex7`:

```
Queue<Client> coada = new Queue<Client>();
coada.Enqueue(new Client("Ana", 2));
coada.Enqueue(new Client("Ion", 1));
coada.Enqueue(new Client("Maria", 3));
Console.WriteLine(string.Join(", ", OrdineTerminare(coada)));
```

Ieșire așteptată:

```
Ion, Ana, Maria
```

Teorie: §7

8. Paranteze corecte

`static bool ParantezeCorecte(string expresie)`: `()`, `[]`, `{}`, fiecare închisă de perechea ei și în ordinea corectă. Orice alt caracter se ignoră.

În `Ex8`:

```
foreach (string e in new string[] { "(a[b]{c})", "(]", "((", ")(", "" })
{
    Console.WriteLine("\"" + e + "\" -> " + ParantezeCorecte(e));
}
```

Ieșire așteptată:

```
"(a[b]{c})" -> True
"(]" -> False
"((" -> False
")(" -> False
"" -> True
```

Teorie: §7

Partea 4 — Când sunt două obiecte „la fel”

9. Punctul

1. Clasa `Punct` cu `int X`, `int Y` și constructorul `Punct(int x, int y)`. Atât.
2. Rulezi codul de mai jos (în `Ex9`) și notezi ieșirea.

3. Adaugi în `Punct` `Equals` și `GetHashCode`: două puncte sunt egale dacă au același `X` și același `Y`; `Equals(null)` și `Equals("x")` întorc `false`, fără excepții; fără `HashCode.Combine` (TEORIE §0). Rulezi din nou.

```
HashSet<Punct> puncte = new HashSet<Punct>();
puncte.Add(new Punct(1, 2));
puncte.Add(new Punct(1, 2));
puncte.Add(new Punct(3, 4));
Console.WriteLine(puncte.Count);
List<Punct> lista = new List<Punct> { new Punct(5, 5) };
Console.WriteLine(lista.Contains(new Punct(5, 5)));
Console.WriteLine(new Punct(1, 2).Equals(null));
Console.WriteLine(new Punct(1, 2).Equals("x"));
```

Ieșire la pasul 2:

```
3
False
False
False
```

Ieșire la pasul 3:

```
2
True
False
False
```

Teorie: §8 · în `RASPUNSURI.txt`: de ce 3 la pasul 2? Ce formulă ai ales pentru `GetHashCode` și ce hash dă ea pentru (1, 2) și (2, 1)?

Partea 5 — LINQ

În partea asta, metodele nu au `for` / `foreach`: doar operatori LINQ.

10. Premianții

`static List<string> Premianti(List<Elev> elevi)`: elevii cu media cel puțin 9, descrescător după medie, iar la medii egale alfabetic. Fiecare rând: nume, spațiu, media cu două zecimale.

În `Ex10`:

```
foreach (string linie in Premianti(Catalog()))
{
    Console.WriteLine(linie);
}
```

Ieșire așteptată:

```
Maria 9.80
Ana 9.50
Vlad 9.50
Dan 9.10
```

Teorie: §10 · `Where`, `OrderByDescending`, `ThenBy`, `Select`, `ToList`

11. Raportul pe clase

`static List<string> RaportClase(List<Elev> elevi)`: câte un rând pe clasă, în ordinea claselor, cu numărul de elevi, media clasei cu două zecimale și elevul cu media cea mai mare.

În `Ex11`:

```
foreach (string linie in RaportClase(Catalog()))
{
    Console.WriteLine(linie);
}
```

Ieșire așteptată:

```
10C: 2 elevi, media 8.45, primul Vlad
11B: 2 elevi, media 8.65, primul Dan
12A: 3 elevi, media 8.07, primul Maria
```

Teorie: §10 · `GroupBy`; un grup e el însuși o colecție, deci merge `g.Count()`, `g.Average(...)`

12. Rescrie 2, 3 și 6 cu LINQ

Câte o variantă LINQ pentru `FaraDubluri`, `Frecvente`, `InAmbele` și `DoarLaPrimul`, cu sufixul `Linq` în nume (`FaraDubluriLinq`…). `Ex12` conține codul din `Ex2`, `Ex3`, `Ex6`, cu metodele noi.

Ieșire așteptată (cele trei, una după alta):

```
Ana, Ion, ana, Maria
a: 3
e: 3
m: 1
n: 1
r: 2
Dan, Ion
Ana, Maria
```

Gata când: ieșirea e identică cu cea de la 2, 3 și 6. În `RASPUNSURI.txt`: care variantă ți se pare mai ușor de citit, la fiecare, și de ce?

Teorie: §10 · `Distinct`, `GroupBy` + `ToDictionary`, `Intersect`, `Except`

13. Întrebări despre catalog

În `Ex13`:

```
List<Elev> cat = Catalog();
Console.WriteLine(___1___);
Console.WriteLine(___2___);
Console.WriteLine(___3___);
Elev lipsa = ___4___;
Console.WriteLine(lipsa == null ? "null" : lipsa.Nume);
Console.WriteLine(___5___);
try
{
    Console.WriteLine(___6___);
}
catch (InvalidOperationException ex)
{
    Console.WriteLine(ex.GetType().Name);
}
```

Fiecare `___n___` e o singură expresie LINQ: **1** există vreun elev cu media sub 5? · **2** toți au media cel puțin 5? · **3** numele primului elev din 11B · **4** primul elev din 9A, fără să arunce · **5** cea mai mare medie, cu două zecimale · **6** numele primului elev din 9A, cu `First`.

Ieșire așteptată:

```
True
False
Ion
null
9.80
InvalidOperationException
```

Teorie: §10 · `Any`, `All`, `First`, `FirstOrDefault`, `Max`

14. Rețeta, nu rezultatul

În `Ex14`:

```
List<int> valori = new List<int> { 5, 20, 8, 30 };
IEnumerable<int> mari = valori.Where(v => v > 10);
List<int> mariAcum = valori.Where(v => v > 10).ToList();
valori.Add(50);
Console.WriteLine(string.Join(" ", mari));
Console.WriteLine(string.Join(" ", mariAcum));
```

Înainte să rulezi, scrie în `RASPUNSURI.txt` ce crezi că afișează. Apoi rulează și explică diferența dintre cele două rânduri.

Teorie: §11

Provocare

15. Cuvântul cel mai des întâlnit

`static string CelMaiFrecventCuvant(string text)`: fără să conteze dacă literele sunt mari sau mici, ignorând `.` `,` `!` `?` de la capetele cuvintelor. La egalitate câștigă primul în ordine alfabetică. Îl scrii de **două ori**, o dată cu bucle și o dată cu LINQ (`CelMaiFrecventCuvantLinq`). `Ex15` afișează întâi rezultatele variantei cu bucle, apoi pe ale variantei LINQ, pe aceleași două texte.

În `Ex15`, pentru varianta cu bucle:

```
string text = "Ana are mere. Ion are pere, ana are si prune!";
Console.WriteLine(CelMaiFrecventCuvant(text));
Console.WriteLine(CelMaiFrecventCuvant("mar par mar par"));
```

Ieșire așteptată (patru rânduri):

```
are
mar
are
mar
```